CHAPTER 8 | CRASH RECOVERY

Sailesh Singh

Introduction

Just like any other electronic device, computer system faces a risk of failure. With such failure, significant information might be lost. Hence, DBMS must ensure proper recovery mechanisms to return the database to the initial state before the failure occurred.

Such backup system should be highly available, and also be continuously updated to keep the most up to date information of the original database.

Failure Classification

There are a large number of failures that can happen in a database.

Among them, the most common ones are:

1. Transaction Failure
    a. Logical Error
    b. System Error
2. System Crash
3. Disk Failure

Transaction Failure

A transaction stops before completion and is rolled back.
It happens due to errors inside the transaction or system problems.

- Logical Error: The transaction itself is wrong.
  Examples: wrong input, invalid calculation, condition fails.
- System Error: The system fails while a transaction is running.
  Examples: power failure, crash, OS failure.

System Crash

- A sudden failure that stops the entire database system.
- Caused by power failure, hardware fault, or OS crash.
- Main memory contents are lost, but disk data remains safe.

Disk Failure

- Failure of the storage device where the database is stored.
- Caused by disk crash, bad sectors, or physical damage.
- Results in loss of database data and requires backup and recovery.

Thus, we have two primary concerns,

1. Identifying the cause of the failure of the disk
2. How the deleted failure mode has affected the contents of the disk.

These algorithms are known as recovery algorithms and consist of two parts:

1. Taking certain actions that can be taken in order to record enough information before failure
2. Use that previously recorded information to restore the correct database state.

What recovery algorithm does is:

- Recovery ensures the database is returned to a correct and consistent state after a failure.
- It uses logs and backups to undo incomplete transactions and redo committed ones.
- It ensures atomicity.
- After recovery:
  - Committed transactions must appear fully executed
  - Failed or incomplete transactions must have no effect on the database

Block Level Data Access (How!)

- Block-level access in database recovery means the DBMS performs recovery operations at the granularity of disk blocks (pages) rather than whole files or individual records.
- All disk reads and writes happen at the block level, making data access efficient and manageable.
- Improves I/O efficiency by reading/writing data in chunks instead of record by record
- Simplifies buffer management, locking, and recovery
- Makes operations like logging, caching, and recovery faster and more reliable

Stable Storage Implementation (Where!)

- Data is stored safely so it is not lost even if failures occur.
- Implemented using multiple disks, data replication, and careful write protocols.
- If one disk fails, data can be recovered from another copy.

Database Recovery Techniques

- They are methods used to restore a database to a correct and consistent state after failures such as system crashes, disk failures, or transaction errors.
- Key ideas:
  - Ensure atomicity and durability (committed changes are not lost)
  - Handle different types of failures: transaction failure, system crash, disk failure
  - Use previously recorded information (logs, backups, or shadow copies) to recover the database

Database Recovery Techniques

- They are methods used to restore a database to a correct and consistent state after failures such as system crashes, disk failures, or transaction errors.
- Key ideas:
  - Ensure atomicity and durability (committed changes are not lost)
  - Use previously recorded information (logs, backups, or shadow copies) to recover the database

Mechanisms of Recovery

Recovery techniques are the safety net that keeps the database consistent and reliable despite unexpected failures. Main approaches include:

1. Log-Based Recovery
   records changes in a log to undo/redo transactions
2. Shadow Paging
   updates are made on a copy; original pages remain safe

Log Based Recovery

A log is a sequence of records that track all operations of a database transaction. Logs are stored in a log file and must be written before the actual database changes. They provide a reliable mechanism to recover data after failures, ensuring data integrity and consistency.

Parts of a Log

1. Start Log: When the transaction begins, a log is created to indicate the start of the transaction.

   Format:<Tn, Start>

   Here, Tn represents the transaction identifier.

2. Operation Log: When the city is updated, a log is recorded to capture the old and new values of the operation.

   Format:<Tn, Attribute, Old_Value, New_Value>

Parts of a Log

3. Commit Log: Once the transaction is successfully completed, a final log is created to indicate that the transaction has been completed and the changes are now permanent.

   Format:<Tn, Commit>

4. Abort/Failure Log: Marks a transaction that failed or was rolled back.

   Format: <Tn, Abort>

Parts of a Log

3. Commit Log: Once the transaction is successfully completed, a final log is created to indicate that the transaction has been completed and the changes are now permanent.

   Format:<Tn, Commit>

4. Abort/Failure Log: Marks a transaction that failed or was rolled back.

   Format: <Tn, Abort>

Example of a log

T0:

read(A);

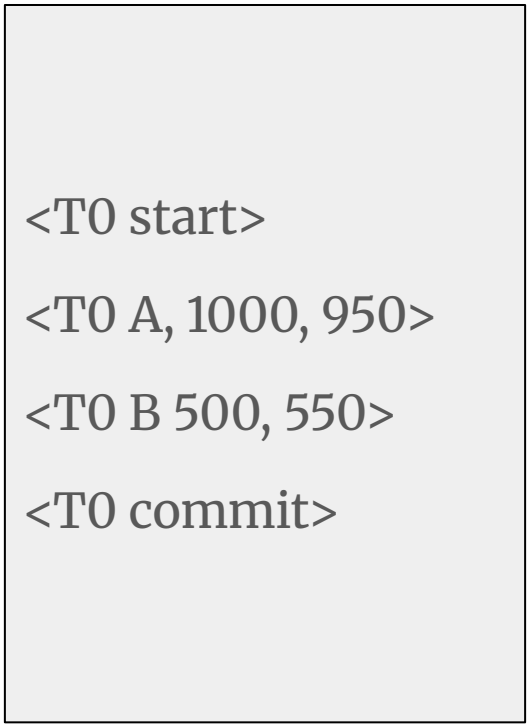
A:= A - 50;

write(A);

read(B);

B := B + 50;

write(B);



Operations in Log-Based Recovery

Undo Operation

The undo operation reverses the changes made by an uncommitted transaction.

Even though the transaction did not commit, its dirty pages may have already been written to disk from the buffer.

Undo restores the old values using the log to remove these partial changes.

Working of the undo operation

Step	Action	Purpose
Trigger	No commit or abort record found.	Identifies a "hanging" or failed transaction.
Operation	Write "Original Value" to the data item.	Reverses the uncommitted changes.
Log Entry	Write special "redo-only" record.	Records the fact that we are restoring the old state.
Final write	Write `<Ti abort>`.	Marks the transaction as officially dead and cleaned up.

Operations in Log-Based Recovery

Redo Operation

The redo operation re-applies the changes of a committed transaction.

Although the transaction has committed, its updated pages may not have been flushed to disk before the crash.

Redo ensures that committed changes are safely applied to the database.

Working of the Redo operation

Phase	Action	Purpose
Analysis	Scan the log for <Ti commit> records.	Determine which transactions must be saved.
Redo Start	Locate the first log record of the oldest committed transaction.	Ensure no updates are missed.
Execution	Set data items to the New Value stored in the log.	"Replay" the history to bring the disk up to date.
Checkpointing	Periodically flush these changes to the actual disk.	Shortens future recovery time by clearing the "to-do" list.

Difference between recovery operations

Undo	Redo
To remove changes from incomplete/failed transactions.	To re-apply changes from successful (committed) transactions.
Uses the Old Value.	Uses the New Value.
Moves the database backward in time.	Moves the database forward in time.
Supports Atomicity.	Supports Durability.

Checkpoints

It is a "bookmark" or a "snapshot" in the database log. It is a point in time where the system stops for a brief second to synchronize its records, making future recovery much faster.

Without checkpoints, a database would have to scan the entire log from the very beginning of time whenever it crashed. With checkpoints, it only has to go back to the most recent "bookmark."

The REDO algorithm

1. Locate the most recent <checkpoint L> record.
2. Set your Undo-List equal to all transactions listed in L.
3. The system starts at the checkpoint and reads every log record forward until it hits the end of the log (the crash point). For every record it finds, it follows these rules:
    a. Format: <Ti, Xj, V1, V2> (Transaction T_i changed item X_j from V_1 to V_2) -> Write the value V_2 to data item X_j.
    b. <Ti start>Action: Add T_i to the Undo-List. This means a new transaction just began.
    c. <Ti commit> or <Ti abort>Action: Remove T_i from the Undo-List. This means T_i finished its job (successfully or otherwise) before the crash, so we don't need to worry about it anymore.

The UNDO algorithm

1. Goto the end of the log file and start backwards.
2. For every record it encounters, it checks if the transaction ID (T_i) is in the Undo-List.
3. If the system finds a record <Ti, Xj, V1, V2> and T_i is in the Undo-List, it performs the "Undo" by writing the old value (V_1) back to the data item X_j.
   It writes a special log record called a CLR (Compensation Log Record). This record basically says: "During recovery, I undid this change."
4. If the system finds <Ti start> and T_i is in the Undo-List, It writes a <Ti abort> record to the log.

Modification Techniques

They define the "agreement" between the RAM and the Hard Drive regarding when data is physically saved.

Since writing to a disk is slow, databases use one of these two strategies to handle updates:

1. Deferred Modification (after committing)
2. Immediate Modification (as changes take place)

Deferred Modification

The database postpones any actual changes to the disk until the transaction is committed. During the Transaction, all changes are kept in a local workspace (RAM) and recorded in the Log. Nothing touches the main database files on the disk.

At Commit, the changes are finally pushed to the disk.

Recovery Needs:

- Redo: Yes (To finish pushing committed work that didn't make it to disk before a crash).
- Undo: No (Since uncommitted work never touched the disk, there is nothing to "undo").

Immediate Modification

The database is allowed to write changes to the disk at any time, even while the transaction is still running.

During the Transaction, changes can be "flushed" to the disk whenever the system feels like it (to free up RAM).

At Commit, the system just makes sure the "Commit" note is in the log.

Recovery Needs:

- Redo: Yes (To ensure committed work is permanent).
- Undo: Yes (Crucial! Because "dirty" uncommitted data likely made it onto the disk before the crash).

Shadow Paging

Shadow Paging is an alternative to log-based recovery.

The database maintains two "Page Tables" (directories that point to where data is physically stored on the disk):

- Shadow Page Table: This is the "Master Copy." It points to the original, consistent state of the database. It is never modified during a transaction.
- Current Page Table: This is the "Working Copy." When a transaction starts, it's an exact clone of the Shadow Table.

Working of Shadow Paging

The Update Process:

- When a transaction wants to change a page (e.g., Page 4), the system finds a new, empty block on the disk.
- It writes the new data into that new block.
- It updates the Current Page Table to point to this new block.
- The Shadow Page Table still points to the old, original Page 4.

If the Transaction Commits: The system simply swaps the tables. The "Current" table becomes the new "Shadow" table. In one instant, all changes become permanent.

Working of Shadow Paging

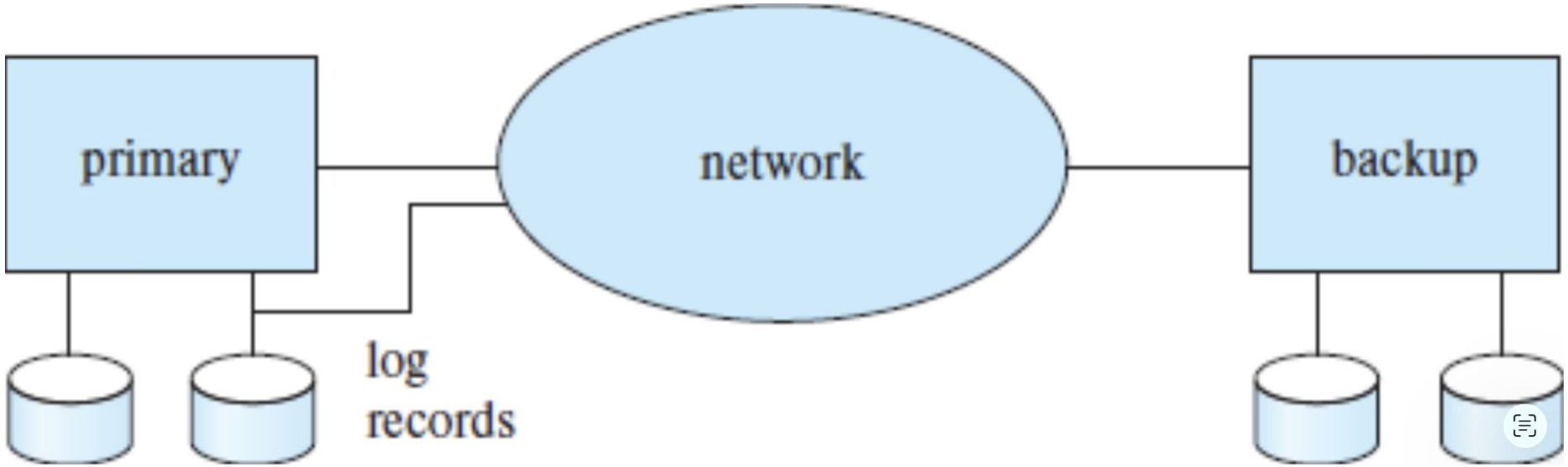
If the System Crashes: Recovery is nearly instant!

- The "Current" table (which was in RAM) is wiped out.
- The system simply reloads the Shadow Page Table from the disk.

Because the Shadow Table still points to the original, clean data, it's as if the unfinished transaction never happened.

Feature	Shadow Paging	Log-Based Recovery
Recovery speed	Very fast (pointer switch)	Slower (redo/undo logs replay)
Transaction commit overhead	High if many pages modified	Low (append log only)
Disk I/O	High for modified pages	Low (sequential log writes)
Complexity	Medium (page table management)	Medium to high (logging, undo/redo)
Crash tolerance	High	High (if logs maintained properly)

High Availability using Remote Backup Systems [19.7]



Components of Remote Backup System

1. The Primary Site (Left Block)

This is your main production server.

Role: It handles all active user transactions, reads, and writes.

Storage: The two cylinders below it represent the Database Data and the Transaction Log.

Action: Every time a change is made, the Primary site writes a record to its local log. As seen by the line extending toward the network, it simultaneously "ships" these log records to the backup site.

Components of Remote Backup System

2. The Network (Center Oval)

This represents the communication link (Internet or a dedicated fiber line) between two physically separate locations.

Role: It acts as the bridge for Log Shipping.

Importance: The speed and reliability of this network determine how "up-to-date" the backup can be. If the network is slow, the backup site might lag behind the primary.

Components of Remote Backup System

3. The Backup Site (Right Block)

This is the "Shadow" or "Standby" server, usually located in a different city or region.

Role: It stays in a state of Continuous Recovery. It receives the log records sent over the network and "replays" them on its own local data copies.

Storage: It maintains its own identical set of disks (Data and Logs).

Action: It monitors the Primary site. If the Primary goes offline, this block "takes over" and becomes the new Primary so that users experience little to no downtime.

Working

- Transactions happen on the Primary.
- Log Records are generated and sent through the Network.
- The Backup receives those records and updates its own disks.
- If the Primary crashes, the Backup is already ready to step in.

What is Big Data?

Massive datasets that can't be processed with traditional tools, characterized by:

- **Volume**: Terabytes to petabytes of data
- **Velocity**: High-speed data generation
- Variety: Structured, semi-structured, unstructured data
- Veracity: Data quality and trustworthiness
- **Value**: Business insights extraction

Why Traditional Databases Fail?

- Can't scale horizontally
- Require fixed schema
- Expensive to scale vertically
- Not designed for unstructured data

Big Data Storage Solutions:

- Distributed File Systems: HDFS, GFS
- NoSQL Databases: MongoDB, Cassandra, Redis
- NewSQL Systems: Google Spanner, CockroachDB